

AGILE TESTING

Michele Marchesi

marchesi@unica.it

**Corso di Ingegneria del Software
A.A. 2025/26**



Introduzione

- Le pratiche che descriveremo in questa sezione sono tipiche dell'eXtreme Programming (XP)
- Sono però adottate **in pratica** da tutti gli sviluppatori agili
- L'XP si basa su alcune pratiche fondamentali
 - Automatizzare tutti i test
 - Test Driven Development (**TDD**)
 - Acceptance Test associati ai requisiti, che sono espressi come User Stories



Test Automatici

- Nell'XP c'è una forte enfasi sul fatto che tutti i test devono essere **automatici**
 - I test stessi sono codice!
- I test devono poter essere fatti girare in qualunque momento, da chiunque con uno sforzo minimo
- Ciò aumenta la confidenza che il nuovo codice aggiunto non destabilizzi quello che c'era prima
 - valore XP del **coraggio**



Test Automatici

- I test automatici possono formare una suite di **regression test**
- Il regression test è la ripetizione dell'esecuzione dei test dopo che sono state cambiate parti di codice
- L'obiettivo dei regression test è **mostrare che il comportamento del sistema non è cambiato**
- I test automatici fanno risparmiare tempo e denaro, perché il *debugging* alla cieca può essere molto lungo e costoso



Benefici dei Test Automatici

- Sviluppo di sistemi più affidabili
- Riduzione dello sforzo per gestire e far girare i test
- Possibilità di effettuare **refactoring** e di modificare i requisiti anche in corso d'opera
- Controindicazioni:
 - **I test non sono gratuiti!**
 - E' necessario trovare un compromesso tra i costi per automatizzare i test e i loro benefici



Test Automation Manifesto (Meszaros et al.)

- Il manifesto contiene le linee guide per creare i test automatici
- I test automatici dovrebbero essere:
 - **Concisi** – Il più semplici possibile, ma non banali
 - **Self Checking** – Devono riportare i risultati senza necessità di un'interpretazione umana
 - **Ripetibili** – Devono poter essere eseguiti ripetutamente senza intervento umano
 - **Robusti** – I test producono sempre gli stessi risultati. Non sono influenzati da cambiamenti dell'ambiente esterno, dalla sequenza in cui sono eseguiti, o dal fatto che un test precedente sia fallito

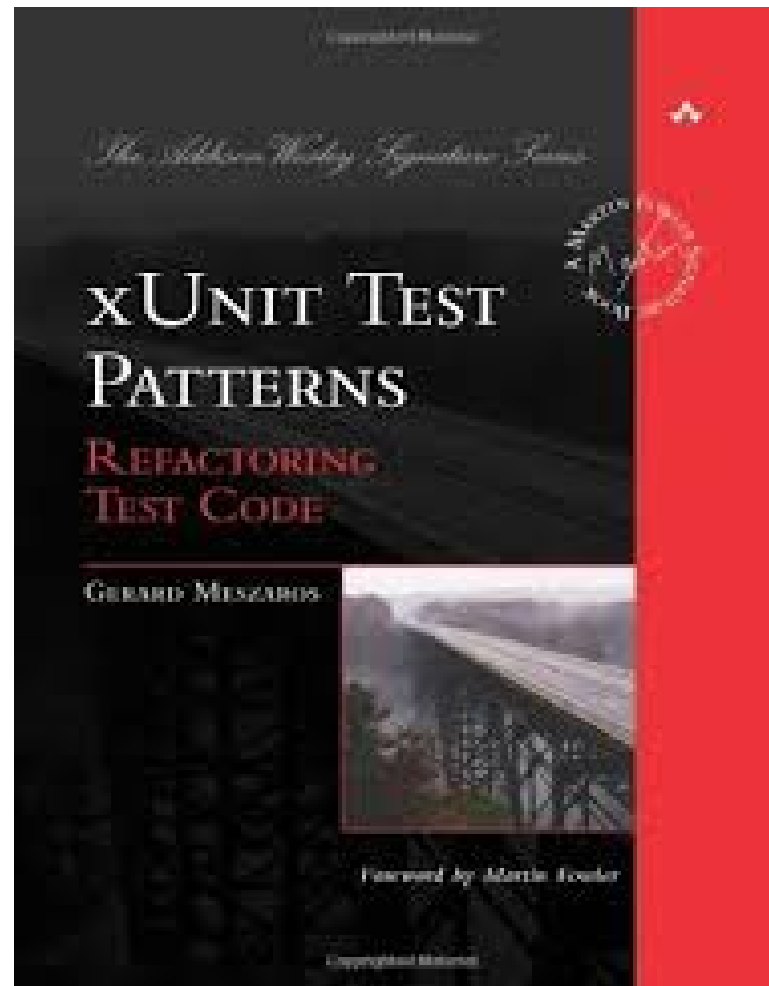


Test Automation Manifesto (Meszaros et al.)

- **Sufficienti** – Devono verificare tutti i requisiti del software sotto test
- **Necessari** – In ogni test, tutto contribuisce alla specifica del comportamento desiderato
- **Chiari** – Ogni istruzione è facile da capire; **il codice dei test contribuisce alla documentazione del software**
- **Efficienti** – I test sono eseguiti in un tempo ragionevole
- **Specifici** – Ogni punto in cui il test può fallire corrisponde ad una funzionalità specifica



XUnit Test Patterns: Refactoring Test Code



Stakeholders

- I tre Stakeholders coinvolti nell'attività di testing automatico sono:
 - Lo **sviluppatore** che deve tener conto del tempo speso per automatizzare i test sia a livello di unit test che di test di accettazione
 - Il **manager** deve concedere agli sviluppatori il tempo per implementare i test. Questo tempo diventa un investimento a lungo termine perché i test diventano uno strumento fondamentale per i test di regressione
 - Il **cliente** deve partecipare alla creazione dei test di accettazione – in Scrum, questo ruolo è svolto soprattutto dal **Product Owner**



Test-Driven Development (TDD)

- **Test First Programming**
- **Prima** di modificare o aggiungere del codice, **scrivere i test** per verificare il codice stesso
- La programmazione guidata dai test è una tecnica che riguarda non solo i test, ma è una **pratica di progetto**
 - Gli sviluppatori iniziano a scrivere i test per una user story e poi scrivono il codice che la implementa
- Se è difficile scrivere un test, è un problema di progetto e non di codifica



Test Driven Development

- Scrivere i test prima del codice
- Tutto il codice deve avere Test Unitari
- Tutto il codice deve superare i test unitari al 100%
- Quando uno o più test falliscono, si corregge il sw sino a che il 100% dei test è superato!
- Quando si trova un bug, si scrivono test che avrebbero scoperto il bug, se fossero stati presenti
- I Test di Accettazione sono eseguiti spesso e se ne pubblicano i risultati
- Tutti i test devono essere automatici!



TDD in 6 passi

- E' una tecnica utilizzata dagli sviluppatori che eseguono rapide iterazioni dei seguenti passi:
 1. Si esegue una breve sessione di progetto, identificando gli oggetti e le loro proprietà rilevanti per sviluppare la funzionalità in esame
 2. Si scrivono alcuni test unitari automatici
 3. Si eseguono tali test, **verificando che falliscano** (non c'è ancora il codice!)



TDD in 6 passi

4. Si implementa giusto il codice che permette di superare i test
5. Si rieseguono tali test, verificando che ora siano superati
6. Si esegue un *refactoring* sul codice e sui test, per migliorarne il funzionamento e l'architettura



TDD in 6 passi: Esempio

Obiettivo: implementare un metodo `somma(a, b)` usando il TDD.

1. Mini design:

- Funzionalità da implementare: sommare due numeri interi.
- Servirà una classe `Calcolatrice` con un metodo `somma(a, b)`.



TDD in 6 passi: Esempio

2. Scrivo un test che fallisce

```
# test_calcolatrice.py
import unittest
from calculatrice import Calcolatrice # modulo da
creare

class TestCalcolatrice(unittest.TestCase):
    def test_somma_due_numeri(self):
        calc = Calcolatrice()
        risultato = calc.somma(2, 3)
        self.assertEqual(risultato, 5)

if __name__ == '__main__':
    unittest.main()
```



TDD in 6 passi: Esempio

3. Il test fallisce

- L'esecuzione darà un errore: ModuleNotFoundError o AttributeError
- Questo è previsto e voluto: il codice ancora non esiste!

4. Scrivo il codice minimo per far passare il test:

```
# calcolatrice.py

class Calcolatrice:
    def somma(self, a, b):
        return a + b
```

5. Il test ora passa

- Rieseguiamo python test_calcolatrice.py
- Il test dovrebbe dare: OK



TDD in 6 passi: Esempio

6. Refactoring

- In questo caso, il codice è già molto semplice. Ma possiamo fare piccoli miglioramenti, ad esempio: Rinominare le variabili in modo più esplicito, Aggiungere un docstring

```
class Calcolatrice:  
    def somma(self, primo_addendo, secondo_addendo):  
        """Restituisce la somma di due numeri."""  
        return primo_addendo + secondo_addendo
```

6+1. Espansione naturale (ciclo TDD continuo). Dopo somma(), si può proseguire così:

- Scrivi un test per sottrai(a, b)
- Fallisce → implementi → test passa → refactoring



TDD: risolve quattro problemi

- **cowboy coding**: imporsi di scrivere test, e di farli superare dal codice sviluppato, evita il c.c.
- **accoppiamento e coesione**: il codice a basso accoppiamento e alta coesione è facile da testare
- **fiducia**: chi scrive codice e supera i test otterrà la fiducia dei propri compagni
- **ritmo**: è facile divagare per ore quando si codifica; se ci si abitua al ritmo: scrivi test, scrivi codice, esegui refactoring, scrivi test... ciò non avviene



TDD: vantaggi concreti

Vantaggio	Descrizione
Design più pulito	Le classi nascono da un bisogno reale (i test)
Meno bug in produzione	I test catturano possibili problemi subito, e sono anche test di regressione
Documentazione viva	I test descrivono come dovrebbe comportarsi il codice
Maggiore confidenza	Si può fare refactoring senza paura
Feedback immediato	Si sa in pochi secondi se si è rotto qualcosa



Test Unitari

- Sono i test scritti per ogni modulo (unità) di codice
- Ogni classe del sistema sviluppato avrà una corrispondente classe di test
- Sono scritti dagli stessi programmatori del codice
- L'insieme dei test unitari è raggruppato in una suite ed i test sono eseguiti in modo automatico
- Devono sempre essere superati al 100%
- Il framework *storico* dei TU è X-Unit



Test di Accettazione

- Sono test di tipo black box, scritti dal cliente o dal Product Owner (PO) in collaborazione con gli sviluppatori
- Sono creati a partire dalla storie (User Stories)
- Sono **di proprietà del cliente o del PO**
- Sono un potente **strumento di documentazione del sistema**
- Quando un test di accettazione passa, il cliente o il PO considera la storia implementata
- La percentuale di TA superati denota lo stato di sviluppo del sistema
- Dovrebbero essere automatici



Test di Accettazione

- Hanno lo stesso template dei test case di tipo black box:
 - Test ID
 - Description, che descrive le precondizioni del test e i suoi passi
 - Expected Results
 - Actual Results



Test di Accettazione: esempio

- Test ID: AC-01
- Description:
 - L'utente visita la pagina di login
 - Inserisce username e password corretti
 - Clicca su "Accedi"
- Expected Result:
 - L'utente viene reindirizzato alla dashboard
- Actual Result:
 - L'utente viene reindirizzato alla dashboard



Testing: Agile vs Tradizionale

Aspetto	Tradizionale	Agile
Quando sono scritti	Dopo lo sviluppo	Prima o durante lo sviluppo (insieme alla user story)
Chi li scrive	Cliente o team QA	Cliente, PO e dev insieme
Frequenza di esecuzione	Alla fine del progetto	In ogni sprint, possibilmente ogni giorno
Automazione	Non sempre presente	Fortemente raccomandata
Ruolo	Verifica finale	Strumento di progettazione + verifica
Legame con user story	Debole o assente	Fortissimo – sono i criteri di accettazione



Riassumendo... test automatici

	Test Unitari	Test di Accettazione
Chi li scrive	Sviluppatori	Cliente, Product Owner (con team)
Comunicazione	Codice e test → comunicano tra sviluppatori	User stories e test → comunicano con il cliente
Ruolo	Verificano il corretto funzionamento interno	Verificano che il comportamento risponda ai requisiti
Documentazione	Documentano il <i>come funziona</i> il codice	Documentano il <i>cosa deve fare</i> il sistema
Benefici	Favoriscono refactoring, sicurezza e proprietà collettiva	Rendono visibili le aspettative e i progressi
Automazione	Sì, eseguiti spesso e in CI	Idealmente automatizzati, spesso integrati in BDD



Principi Agili applicati al testing

- Customer collaboration → coinvolgere il cliente o il Product Owner nei test
- Working software → test = misura della qualità
- Responding to change → test flessibili e aggiornabili
- Individuals and interactions → comunicazione continua nel team

